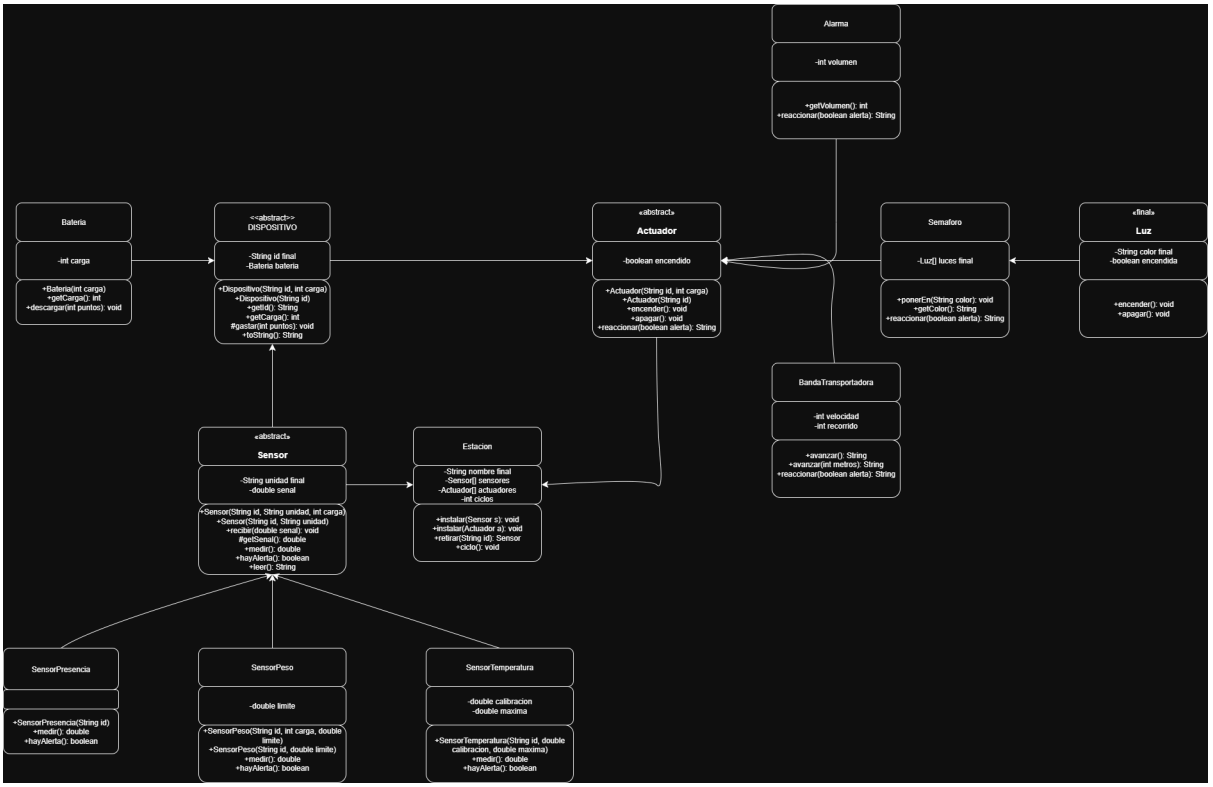
# Integrantes

- Nombre: David Uriel Villarruel Sánchez
- Número de cuenta: 323272733

# 1. Diagrama UML



## 2.-Salida de `Main`

```
PS C:\Users\OEM\mpoo\mpoo-2027-1\entregas\villarruel_david\p08> java -cp out estacion.Main
Instalado: Sensor S1 (100%) presencia
Instalado: Sensor S2 (80%) peso, limite 25.0
Instalado: Sensor S3 (100%) temperatura, max 45.0
Instalado: Actuador A1 (100%) banda, 2 m por ciclo
Instalado: Actuador A2 (100%) semaforo en apagado
Instalado: Actuador A3 (100%) alarma, volumen 10
== Ciclo 1 en Anden 3
  S1 mide 12.0 cm
  S2 mide 18.5 kg
  S3 mide 40.0 grados
  A1 avanza 2 m, lleva 2 m
  A2 en verde
  A3 en silencio
== Ciclo 2 en Anden 3
  S1 mide 9.5 cm
  S2 mide 32.0 kg ALERTA
  S3 mide 42.0 grados
  A1 se detiene
  A2 en rojo
  A3 suena a volumen 10
Retirado: S2
== Ciclo 3 en Anden 3
  S1 mide 15.0 cm
  S3 mide 46.5 grados ALERTA
  A1 se detiene
  A2 en rojo
  A3 suena a volumen 10
Regresa: Sensor S2 (78%) peso, limite 25.0
Instalado: Sensor S2 (78%) peso, limite 25.0
A1 avanza 5 m, lleva 7 m
Anden 3: 3 sensores, 3 actuadores, 3 ciclos
PS C:\Users\OEM\mpoo\mpoo-2027-1\entregas\villarruel_david\p08>
```

## 3.- Respuestas

### 1. Sobrecarga contra sobrescritura

La sobrecarga ocurre cuando una clase tiene métodos con el mismo nombre, pero con diferentes parámetros. Por ejemplo, en `BandaTransportadora` tenemos `avanzar()` y `avanzar(int metros)`. El primero utiliza la velocidad configurada y el segundo permite indicar cuántos metros avanzar.

La sobrescritura ocurre cuando una subclase vuelve a implementar un método heredado para darle un comportamiento específico. Por ejemplo, `SensorPeso` sobrescribe el método `medir()` de `Sensor` para convertir la señal recibida a kilogramos.

La diferencia es que la sobrecarga permite utilizar el mismo nombre con distintos parámetros, mientras que la sobrescritura cambia el comportamiento de un método heredado.

## 2. Composición contra agregación

La `Bateria` es composición porque `Dispositivo` crea su propia batería mediante `new Bateria(carga)` y la administra internamente. La batería forma parte del dispositivo.

En cambio, `Estacion` tiene agregación con los sensores porque recibe objetos que ya fueron creados y los guarda en su arreglo. No crea los sensores dentro de `instalar(Sensor sensor)`.

Esto se observa con S2: en el ciclo 3 se retira de la estación, pero el objeto sigue existiendo y conserva su carga del 78 %. Después se puede volver a instalar. Por eso, el sensor puede existir independientemente de la estación.

## 3. Modificadores de acceso

La línea `bascula.gastar(5);` produce un error de compilación porque `gastar()` está declarado como `protected` en la clase `Dispositivo`.

`SensorPeso` sí puede utilizar ese método porque hereda de `Sensor` y, a través de ella, de `Dispositivo`. En cambio, `Main` está en el paquete `estacion` y no hereda de `Dispositivo`, por lo que no puede acceder directamente a ese método protegido.

4.-ERROR

```
PS C:\Users\OEM\mpoo\mpoo-2027-1\entregas\villarruel_david\p08> javac -d out (Get-ChildItem src -Recurse -Filter *.java).FullName
C:\Users\OEM\mpoo\mpoo-2027-1\entregas\villarruel_david\p08\src\estacion\Main.java:22: error: gastar(int) has protected access in Dispositivo
        bascula.gastar(5); // LINEA TEMPORAL PARA PROVOCAR EL ERROR
                ^
1 error
PS C:\Users\OEM\mpoo\mpoo-2027-1\entregas\villarruel_david\p08>
```